200 COP in MI: The Case for Analysing Toy Language Models

by **Neel Nanda** 28th Dec 2022

This is the second post in a sequence called 200 Concrete Open Problems in Mechanistic Interpretability. Start here°, then read in any order. If you want to learn the basics before you think about open problems, check out my post on getting started. Look up jargon in my Mechanistic Interpretability Explainer

***Disclaimer**: Mechanistic Interpretability is a small and young field, and I was involved with much of the research and resources linked here.* Please *take this sequence as a bunch of my personal takes, and try to seek out other researcher's opinions too!*

Motivation

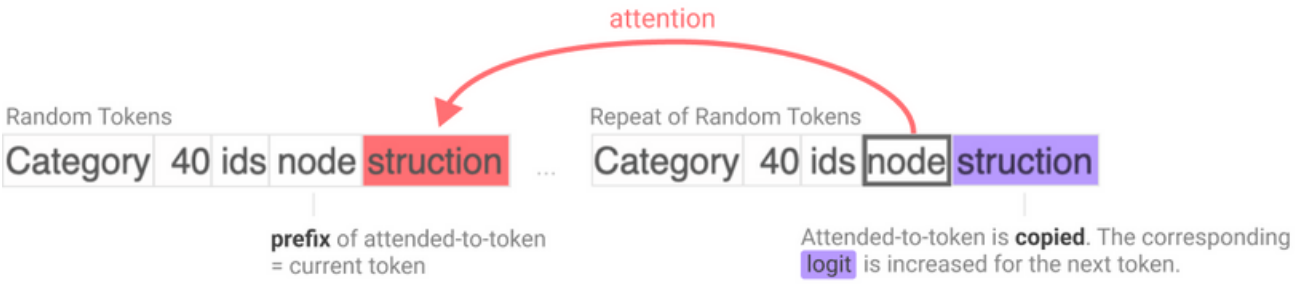
In A Mathematical Framework for Transformer Circuits, we got a lot of traction interpreting toy language models - that is, transformers trained in exactly the same way as larger models, but with only 1 or 2 layers. I think there's a lot of low-hanging fruit left to pluck when studying toy language models! To accompany this piece, **I've trained and open sourced some toy models.** The models are documented here: there are 12 models, 1 to 4 layers, one attention-only, one normal (with MLPs and GELU activations) and one normal with SoLU activations.

So, why care about studying toy language models? The obvious reason is that **it's *way* easier to get traction**. In particular, the inputs and outputs of a model are intrinsically interpretable, and in a toy model there's just not as much space between the inputs and

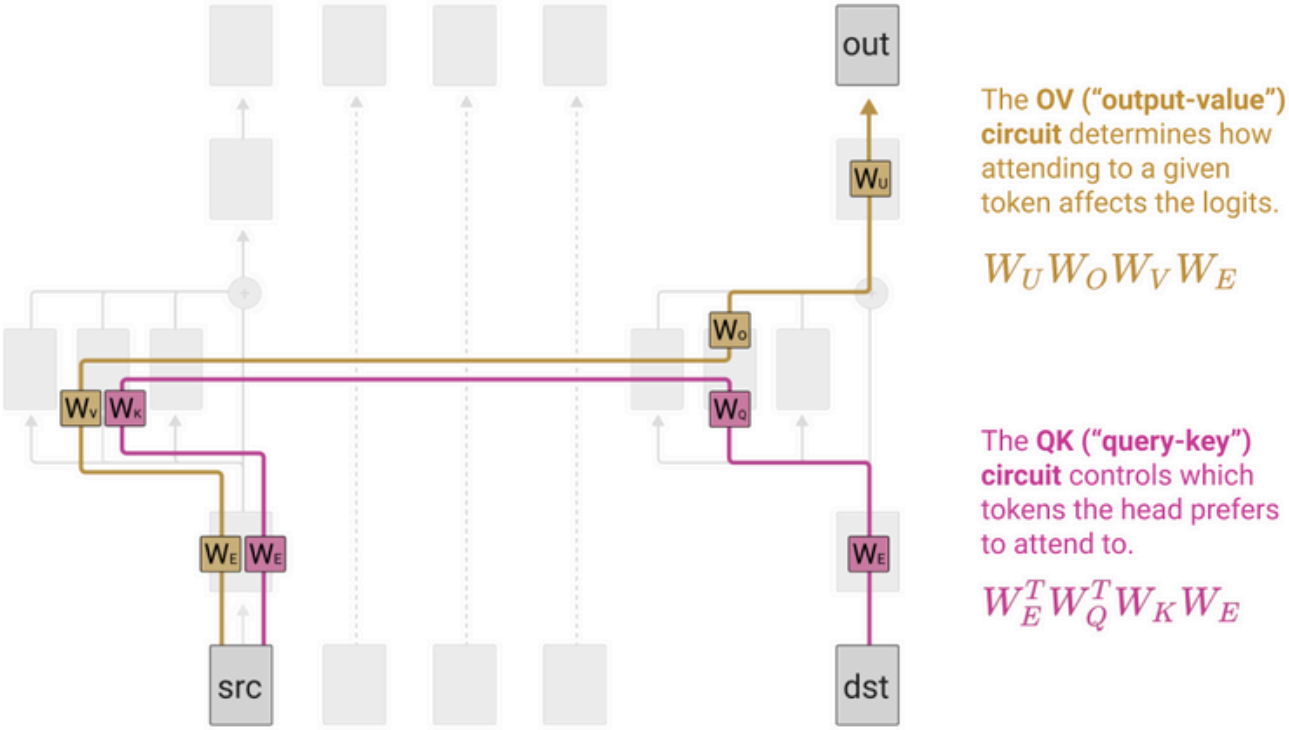
outputs for weird complexity to build up. But the obvious objection to the above is that, ultimately, we care about understanding real models (and ideally extremely large ones like GPT-3), and learning to interpret toy models is not the actual goal. This is a pretty valid objection, but to me, there are two natural ways that studying toy models can be valuable:

The first is by finding fundamental circuits that recur in larger models, and **motifs** that allow us to easily identify these circuits in larger models. A key underlying question here is that of **universality**: does each model learn its own weird way of completing its task, or are there some fundamental principles and algorithms that all models converge on?

A striking example of universality is **induction heads**, which we found in A Mathematical Framework in two layer attention-only models. Induction heads are part of a two head induction circuit which models use to detect and continue repeated sequences from earlier in the prompt, and which turn out to be such a fascinating circuit that we wrote a whole other paper on them! They're **universal in all models we've looked at**, they all appear in a sudden phase change, they seem to be a core mechanism behind complex behaviours such as translation and few shot learning, and they seem to be the main way that transformers use text far back in the context to predict the next token better. (See my overview of induction circuits for more). And knowing about these has helped to disentangle the more complex behaviour of indirect object identification.



The second is by forming a better understanding of ***how* to reverse engineer models** - what are the right intuitions and conceptual frameworks, what tooling and techniques do and do not work, and what weird limitations. This one feels less clear to me. Our work in A Mathematical Framework significantly clarified my understanding of transformers in general, especially attention, in a way that seems to generalise - in particular, thinking of the residual stream as the central object, and the significance of the QK-Circuits and OV-Circuits. But there's also ways it can be misleading, and some techniques that work well in toy models seem to generalise less well.



One angle I'm extremely excited about here is **reverse engineering MLP neurons in tiny models** - our understanding of transformer MLP layers is still extremely limited and there are confusing phenomena we don't understand, like superposition and polysemanticity. And we don't yet have even a *single* published example of a fully understood transformer neuron! I expect I'd learn a lot from seeing neurons in a one or two layer language model be reverse engineered.

My personal guess is that the lessons from toy models generalise *enough* to real models to be worth a significant amount of exploration, combined with careful testing of how much the insights do in fact generalise. But overall I think this is an important and non-obvious scientific question. And being proven wrong would also teach me important things about transformers!

Resources

- **Demo:** Exploratory Analysis Demo is a walkthrough of how to use basic mechanistic interpretability techniques in my TransformerLens library. The notebook explores an unfamiliar task in GPT-2 Small, but the same techniques transfer to these toy models!
  - **I recommend copying this notebook and using this as a starting point - don't start from scratch!** (Change the model name from `gpt2-small` to `solu-1l`, `attn-only-2l` etc)
- A video walkthrough I made on A Mathematical Framework for Transformer Circuits

- TransformerLens contains 12 toy models - attention-only, normal (with MLPs) and SoLU (with MLPs) transformers with 1, 2, 3 or 4 layers. Documented here
  - Load them with `HookedTransformer.from_pretrained('solu-1l')` or `gelu-1l` or `attn-only-1l` (etc for more layers)
- Neuroscope.io - a website I made which shows the text that most activates each neuron in several SoLU language models I trained, including the toy SoLU models mentioned above. (Under construction!)
- My Explainer, especially the sections on Transformers, A Mathematical Framework, Induction Circuits and Mechanistic Interpretability Techniques

Tips

- The structure of a good research project is mostly to identify a problem or type of text that a toy model can predict competently and then to reverse engineer how it does it.
- There are a *lot* of behaviours to explore here, and I've only thought of a few! In particular, my toy models were trained 20% on Python code which is much more structured than natural language, I recommend starting here!
- Once you've found a good problem, it's good to be extremely concrete and specific.
  - Spend some time just inputting text into the data and inspecting the output, editing the text and seeing how the model's output changes, and exploring the problem.
    - Importantly, try to find inputs where the model *doesn't* do the task - it's easy to have an elaborate and sophisticated hypothesis explaining a simple behaviour.
  - Find a clean, concrete, minimal input to study that exhibits the model behaviour well.
    - Good examples normally involve measuring the model's ability to produce an answer consisting of a single token.
      - It's significantly harder to study why the model can predict a multi-token answer well (let alone the loss on the entire prompt), because once the model has seen the first token of the prompt, producing the rest is *much* easier and may require some other, much simpler, circuits. But the first token might also be shared between answers!
    - It's useful to explore problems with two answers, a correct and incorrect one, so you can study the difference in logits (this is equal to the difference in log prob!)

        - It's useful to be able to compare two prompts, as close together as possible (including the same number of tokens), but with the correct and incorrect answers switched. By setting up careful counterfactuals like this, we can isolate what matters for *just* the behaviour we care about, and control for model behaviour that's common between the prompts.
        - A good example would be comparing how the model completes "The Eiffel Tower is in the city of" with Paris, while it follows Colosseum with Rome.
            - By studying the logit difference between the Rome and Paris output logits rather than just the fact that it outputs Paris, we control for (significant but irrelevant) behaviour like "I should output a European capital city" (or even that " Paris" and " Rome" are common tokens!)
            - By using techniques like activation patching, we can isolate out *which* parts of the model matter to recall factual knowledge.
- To investigate the problem, the two main tools I would start with are direct logit attribution (which identifies the end of the circuit and works best in late layers) and activation patching (which works anywhere)
    - Note that in an attention only model, the *only* thing that final layer heads can do is affect the output, so direct logit attribution is particularly useful there, as the final layer heads are also likely to be the most interesting

Problems

This spreadsheet lists each problem in the sequence. You can write down your contact details if you're working on any of them and want collaborators, see any existing work or reach out to other people on there! (thanks to Jay Bailey for making it)

- Understanding neurons
    - **B-C* 1.1 -** How far can you get with really deeply reverse engineering a neuron in a 1 layer (1L) model? (solu-1l, solu-1l-pile or gelu-1l in TransformerLens)
        - 1L is particularly easy, because each neuron's output adds directly to the logits and is not used by anything else, so you can directly see how it is used.
        - **B* 1.2 -** Find an interesting neuron in the model that you *think* represents some feature. Can you fully reverse engineer which direction in the model should activate that feature (ie, as calculated from the embedding and attention, in the residual stream in the middle of the layer) and compare it to the neuron input direction?

        - **B* 1.3 -** Look for trigram neurons - eg "ice cream -> sundae"
            - Tip: Make sure that the problem can't easily be solved with a bigram or skip trigram!
        - **B* 1.4 -** Check out the SoLU paper for more ideas. Eg, can you find a base64 neuron?
    - **C* 1.5 -** Ditto for 2L or larger models - can you rigorously reverse engineer a neuron there?
    - A-B 1.6 - Hunt through Neuroscope for the toy models and look for interesting neurons to focus on.
    - A-B 1.7 - Can you find any polysemantic neurons in neuroscope? Try to explore what's up with this
    - B 1.8 - Are there neurons whose behaviour can be matched by a regex or other code? If so, run it on a ton of text and compare the output.
- **B-C* 1.9 -** How do 3-layer and 4-layer attention-only models differ from 2L?
    - In particular, induction heads were an important and deep structure in 2L Attn-Only models. What structures exist in 3L and 4L Attn-Only models? Is there a circuit with 3 levels of composition? Can you find the next most important structure after induction heads?
    - **B* 1.10 -** Look for composition scores; try to identify pairs of heads that compose a lot
    - **B* 1.11 -** Look for evidence of composition. E.g. one head's output represents a big fraction of the norm of another head's query, key or value vector
    - **B* 1.12 -** Ablate a single head and run the model on a lot of text. Look at the change in performance. Find the most important heads. Do any heads matter a lot that are *not* induction heads?
- **B-C* 1.13 -** Look for tasks that an nL model cannot do but a (n+1)L model can - look for a circuit! Concretely, I'd start by running both models on a bunch of text and looking for the biggest differences in per-token probabiliy
    - **B* 1.14 -** How do 1L SoLU/GELU models differ from 1L attention-only?
    - **B* 1.15 -** How do 2L SoLU models differ from 1L?
    - B 1.16 - How does 1L GELU differ from 1L SoLU?
- **B* 1.17 -** Analyse how a larger model "fixes the bugs" of a smaller model
    - **B* 1.18 -** Does a 1L MLP transformer fix the skip trigram bugs of a 1L Attn Only model? If so, how?

- Does a 3L attn only model fix bugs in induction heads in a 2L attn-only model? Possible examples (make sure to check that the 2L *can't* do this!):
  - **B* 1.19 -** Doing split-token induction: where the current token has a preceding space and is one token, but the earlier occurrence has *no* preceding space and is two tokens. (Eg " Claire" vs "Cl|aire")
  - B 1.20 - Misfiring when the previous token appears multiple times with different following tokens
  - B 1.21 - Stopping induction on a token that likely shows the end of a repeated string (eg . or ! or ")
- B 1.22 - Ditto, does a 2L model with MLPs fix these bugs?

- A-C 1.23 - Choose your own adventure: Just take a bunch of text with interesting patterns and run the models over it, look for tokens they do really well on, and try to reverse engineer what's going on - I expect there's a lot of stuff in here!

Previous:

200 Concrete Open Problems in Mechanistic Interpretability: Introduction

No comments 108 karma

Next:

200 COP in MI: Looking for Circuits in the Wild

3 comments 16 karma

Mentioned in

41 200 Concrete Open Problems in Mechanistic Interpretability: Introduction
36 A circuit for Python docstrings in a 4-layer attention-only transformer
14 AISC 2024 - Project Summaries
14 Mechanistic Interpretability Quickstart Guide
18 200 COP in MI: Exploring Polysemanticity and Superposition

Load More (5/7)

2 comments, sorted by top scoring

[-] **redhatbluehat** 3y ▼ 0 ▲ ✕ 0 ✓

Hi Neel! Thanks so much for all these online resources. I've been finding them really interesting and helpful.

I have a question about research methods. "How far can you get with really deeply reverse engineering a neuron in a 1 layer (1L) model? (solu-1l, solu-1l-pile or gelu-1l in TransformerLens)."

I've loaded up solu-1l in my Jupyter notebook but now feeling a bit lost. For your IOI tutorial, there was a very specific benchmark and error signal. However, when I'm just playing around with a model without a clear capability in mind, it's harder to know how to measure performance. I could make a list of capabilities/benchmarks, systematically run the model on them, and then pick a capability and start ablating the model and seeing effect on performance. However, I'm then restricted to these predefined capabilities. Like, I'm not even sure what the capabilities of solu-1l are.

I could start feeding solu-1l with random inputs and just "looking" at the attention patterns. But I'm wondering if there's a more efficient way to do this-- or another strategy where research does feel like play, as you describe in your notebook.

Thank you!

[−] **Neel Nanda** 3y ▼ 1 ▲ ✕ 0 ✓

Great question! My concrete suggestion is to look for interesting neurons in Neuroscope, as I discuss more in the final post°. This is a website I made that shows the text that most activates each neuron in the model (for a ton of open source models), and by looking for interesting neurons, you can hopefully find some hook - find a specific task the model can consistently-ish do, analogous to IOI (with a predictable structure you can generate prompts for, ideally with a somewhat algorithmic flavour - something you could write code to solve). And then do the kinds of things in the IOI notebook. Though note that for a 1L model, you can actually mechanistically look at the weights and break down what the model is doing!

On a meta level, the strategy you want to follow in a situation like this is what I call **maximising surface area**. You want to explore things and try to get exposed to as many random details about the model behaviour as you can. So that you can then serendipitiously notice something interesting and dig into it. The meta-lesson is that when you feel stuck and meandering, you want to pick *some* purpose to strive for, but that purpose can just be "put yourself in a situation where you have so much data and context that you can spontaneously stumble across something interesting, and cast a really wide net". Concretely, you want to look for some kind of task/capability that the model is capable of, so you can then try to reverse-engineer it. And a good way to do this is just to run the model on a bunch of dataset examples and look at what it's good at, and see if you can find any consistent patterns to dig into. To better explore this, I made a tool to visualise the top 10 tokens predicted for each token in the text in Alan Cooney's CircuitsVis library. You can filter for interesting text by eg looking for tokens where the model's log prob for the correct next token is significantly higher than attn-only-1l, to cut things down to where the MLPs matter (I'd cut off the log prob at -6 though, so you don't just notice when attn-only-1l is really incorrect lol).

Moderation Log

More from Neel Nanda

19 Building and evaluating model diffing agents bilalchughtai, Josh Engels, Neel Nanda 6h 0

32	Models May Behave Worse When Eval Aware…	Senthooran Rajamanoharan, Neel …	2d	0
65	models have some pretty funny attractor sta…	aryaj, Senthooran Rajamanoharan, …	3mo	0

View more

Curated and popular this week

39	Sympathy for both sides of the egregious misalignment debate	Steven Byrnes	7h	5
26	A Mike's-Eye View of ARC's Research	Mikewins	3d	0
19	Building and evaluating model diffing agents	bilalchughtai, Josh Engels, Neel Nanda	6h	0